忆时塔（Chronomem）实证评估：召回保持、压缩行为与递归总结的边界

YSTYLE[1] · ORCID：0009-0009-6929-0070 · lxy5266@live.com
DeepSeek V4.1 Flash[2] · AI 共同作者

[1] *position 论文（AiraXiv 2610.0035）的实证配套。实现、评估工具与原始结果归档见 `https://atomgit.com/ystyle/chronomem`。本文为中文审阅版；**投稿以英文版为准**。* [2] *AI 共同作者（与 AiraXiv 预印本的署名一致）。详见文末 AI 使用披露。*

摘要

忆时塔的 position 论文提出了一份评估计划，但没有给出任何测量。本文把它执行了。评估对象是一个时间分层的外部记忆服务——它区分**精确回想**（沿引用链走到原始条目，不调用 LLM）与**语义回想**（读 LLM 生成的窗口摘要）——本文沿三条轴线评估：召回如何随事实的年龄退化、递归总结链保留了多少信息、以及系统在两个月真实使用中究竟记住了什么。

三个发现。**（1）近因基线的失效是结构性的，不是渐进的。** 有界近因窗口在滞后一天时满分，随后直接归零；对六档窗口的消融显示断崖随容量移动（20 条 → 1 天；200 条 → 7 天）。一次性压缩在每一档滞后都是零分，**包括最近的那一段**。分层的精确回想在整个 14 天跨度上保持 100%，且跨 seed 零方差。

（2）递归步的保真度取决于输入多样度，而非层级。 用正确的口径测量（同一项目、同一 7 天窗口）显示：当某项目在该周只有一天的摘要时，周摘要与其输入**逐字节相同**（755–1112 字符文本上实测相似度 1.0000）——90% 的数量保留、0.94 倍长度比，即**根本没有压缩**。输入达到两天及以上时该步才真正压缩（0.53 倍），数量保留降到 58%、因果连接词降到 48%。糊在一起平均时，单天样本会把保真度从 58% 抬到 81%。

（3）两个月真实使用中，活下来的是名字，不是数字。 一项盲测——agent 只被允许查阅记忆服务、不得访问代码仓库——**6/6 答对了**预先准备的问题，全部经由无 LLM 的路径，且包含精确数字。但沿摘要链上升时，最先牺牲的是量化证据与因果机制，专有名词与标识符存活最久。本文还报告了两个测量陷阱，它们在被发现前都产出了貌似可信的错误结果：一个长度假象让**更粗的层级看起来更保真**，一个分组错误让单天输入主导了平均值。

最后，我们报告一个关于评估方法本身的负面结果：独立的 LLM 判定者给出了具体主张，但这些主张经不起核验——我们深入核验的两个主张**都是错的**，且都朝**夸大丢失**的方向偏。因此本文结论建立在机械的、可复现的测量之上，判定者的输出只用于指示该往哪里看。

1. 引言

position 论文把忆时塔介绍为一个时间分层的外部记忆服务，并陈述了它的核心设计主张：**回想不是单一操作**。**精确回想**沿物化引用链走到原始条目，不调用 LLM；**语义回想**读 LLM 生成的摘要，是唯一真正压缩粗窗口的形态。它以一份评估计划收尾，没有任何结果。

本文报告这些结果。该计划有三部分，逐一回应：

- **（i）** 与扁平记忆和一次性压缩的受控对照，考察召回准确率与 token 成本；
- **（ii）** 沿摘要链的语义漂移研究；
- **（iii）** 在作者自己的持续部署上考察工作日志用例。

有两点使这份评估不同于一般的合成基准练习。其一，第（iii）部分跑在一个已持续积累约两个月真实记录的部署上，语料不是编造的。其二，在执行第（ii）部分时我们发现，**最直觉的测量方式是错的，且错到会反转结论**——我们把修正的细节一并报告，因为测量方法本身是结果的一部分。

我们还报告一个方法层面的负面结果（§6）。我们在同一批材料上跑了独立的 LLM 判定者；在我们深入核验的两个具体主张中，**两个都错了**，且都高估了信息丢失。这不是说判定者无用——它们指向了两个真实且值得追查的现象——但它确实意味着它们的计数不可引用，也正是本文全程依赖机械测量的原因。

1.1 贡献清单

1. 一次覆盖**五条臂**的受控对照——近因、一次性压缩、精确下钻、天级摘要与**周级摘要**——表明近因基线的退化是**容量断崖**而非渐进衰减，并由六点消融验证（§3）。
2. 对**递归总结步**的修正测量，表明其保真度由*输入多样度*决定：单输入步根本不压缩、甚至可能与输入逐字节相同；多天步把文本减半并丢掉约 40% 的数量（§4）。
3. 在真实数据上刻画**什么能沿链存活**：标识符活得比因果连接词久，因果连接词活得比量化证据久（§5）。
4. 一项盲测证实**无 LLM 路径足以精确召回**数月前的具体操作性事实，评分依据是独立建立的标准答案（§5.1）。
5. 一个关于 LLM 判定者的**方法论负面结果**，以及两个带修正记录的测量陷阱（§6）。

---

2. 方法

2.1 两份刻意不同的语料

合成 trace。 一个带 seed 的生成器（`bench/gen_trace.py`）产出 N 个模拟日的 agent 活动：每天产生少量**事实**（一条带记录结果的决策）加上填充活动，另有一套探针问题询问更早日子里的事实。开发过程中由测量确立的约束：

- **答案按轮转分配**，而非均匀随机：小探针集上的均匀抽样会聚集，9 个探针里曾有一个选项占 7 个（78% 的多数类基线，不留任何检出真实差异的余地）。轮转把该基线压到约 1/|选项数|。
- **每天的主题不重复抽取。** 同一天重复一个主题会让「第 N 天 X 选了什么」产生歧义；54 个（天，主题）组合中有 6 个歧义，而它们解释了分层后端**每一次**失分——系统确实记住了事实，是抽取器取了另一个。
- **探针永远回看，且滞后随运行增长。** 当天探针会被任何近因窗口答出（20 条窗口得 7/9）；固定的短滞后同样会被任何有界窗口答出（在 30 天 trace 上，1 天滞后时 20 条窗口仍得 90.6%）。因此滞后随日号增长，而这正是分层存储应当取胜的那条轴。
- 每个事实在文本里携带一个**不透明的引用号**（`ref ZQ01`）。摘要会逐字保留它，使抽取无歧义——按（主题，天）定位并不可靠，因为摘要的散文结论惯常把多个主题与多个选项写进一句话。

部署语料。 作者自己的忆时塔部署，自 2026-08-29 起连续运行，单用户。评估时它持有 480 条工作日志、1201 条 agent 活动记录与 63 条教训记录，工作日志有 37 个天级与 6 个周级摘要窗口。该语料没有标

准答案，因此用于**存活**测量（§4、§5）而非准确率。

2.2 五条吃同一份输入的后端

后端	机制
`flat-recency`	保留全部，但只暴露最近的 20 条
`one-shot-compaction`	每 2 天把历史重写成一份运行简报，丢弃细节
`chronomem-exact`	沿引用链下钻到原始条目（不调用 LLM）
`chronomem-day`	读**天级**摘要，即事实自己所在窗口的那一条
`chronomem-week`	读包含该事实的**周级**摘要

五条臂吃同一份 trace、答同一组探针，并由**同一个抽取器**读取——弱抽取器会同等压低所有臂，从而保住对照；而每条臂各配一个读取器会把两者混淆。

一条曾经缺失的臂，以及它为何要紧。 我们第一版对照**没有粗粒度摘要臂**：摘要臂只读*天级*摘要。对一篇主张关乎*分层*的论文，这是实打实的缺口——被命名为「分层」的那一侧只测了一层，而粗层——那个本该以信息换跨度的东西——**从未被提问过**。因此我们补了第五条臂 `chronomem-week`，它从包含该事实的周级摘要作答。

两条摘要臂不是同一测量的两种粒度。天级臂读事实所在的窗口，问题与材料范围一致；周级臂读一个覆盖另外六天的窗口，答案是被压缩后的若干项之一。这个不对称恰恰是架构所主张的取舍，只有两条臂都在才看得见。

2.3 「滞后」的含义，以及为何分桶是区间

探针按滞后（事实与提问之间的天数）分桶。桶边界是 0/1/2/3/5/7/14/30/60，所以标着 7 的桶实际容纳滞后 7–13 天，而 30 天 trace 上最后一桶包含 29 天的探针。表格报告真实区间（`7-13`、`14-29`）：把下界当成滞后本身会引出数据并不支持的容量结论。

2.4 测量这条链：三次尝试，两次是错的

这一点值得明写，因为错误版本产出过可信、看起来可发表数字。

尝试 1——整天对整周。 保留率算出来是 81–84%，暗示粗层几乎无损。这不是有效比较：一个周包覆盖 5–7 天、比天包长约 3.4 倍，所以它既包含被测当天，又包含另外六天。目标文本更大，某个 token 就更有可能出现，而这与压缩无关。在一个把整天对整周的变体里，这个假象强到让**更粗的层级看起来比更细的还保真**（数量上是 42% 对 29%）——一个本应立即引起怀疑的结果。

尝试 2——单独一天对该周的对应项目段。 仍不对等：周段覆盖整周。

尝试 3（本文采用）。 把两侧都限制到**同一项目、同一 7 天窗口**——这才是递归真正做的变换，因为总结器按 group key 聚合某个项目的子摘要。只有到这一步，真正要考察的效应才显现（§4）。

2.5 机械的 token 存活

为避免评判散文，存活是**按类别统计 token** 来测量的：

- **数量**——两位及以上的数字、比例、小数（年份作为标签排除）；

- **符号**——反引号标识符、`--flag`、驼峰与下划线命名；
- **因果**——标记推理的连接词（因为／由于／根因／导致／因此／所以／退化成／…）。

对每一层，我们报告更细层 token 仍存在的比例。这是客观且可复现的。它不判断人类是否会说某个事实「存活」了，它测量的是信息的**具体载体**是否还在。

全文严格区分 `absent`（该层存在但略去了该事实）与 `missing`（该窗口根本没有摘要）。把二者混为一谈会把「未生成」报成信息丢失——开发过程中这个错误犯过两次，一次在此处，一次在更早的漂移研究中。

2.6 隔离

合成实验会写入，因此其客户端在**构造时拒绝**生产端口。真实语料读取器则**默认指向生产**——真实语料只存在于那里——因此它只允许只读工具，且这一点由代码强制而非仅写在文档里。开发过程中曾有一个默认指向生产，导致一份合成 trace 被写进线上存储；它已被移除，并加上了这道守卫。

---

3. 召回保持：基线的失效是结构性的

3.1 设置

30 天合成 trace，300 事件、32 探针、三个 seed（7/11/23）。按滞后桶统计准确率并对 seed 取平均；`±` 是跨 seed 标准差。图 1 画出形状，表格给出数字。

3.2 结果

滞后（天）	近因窗口	一次性压缩	精确	天级摘要	周级摘要
1	100%	0%	**100%**	100%	0%
2	0%	0%	**100%**	50%	0%
3–4	0%	0%	**100%**	25%	0%
5–6	0%	0%	**100%**	25%	0%
7–13	0%	0%	**100%**	57.1%	28.6%
14–29	0%	0%	**100%**	66.7%	33.3%
整体	6.2%	**0.0%**	**100.0%**	**53.1%**	**18.8%**

与之前一致，精确回想在每一桶都是 100%。两条摘要臂是单次运行（单个 seed），且每桶的探针数只有 2–14 个，所以**分桶数字噪声大，总数更可信**。

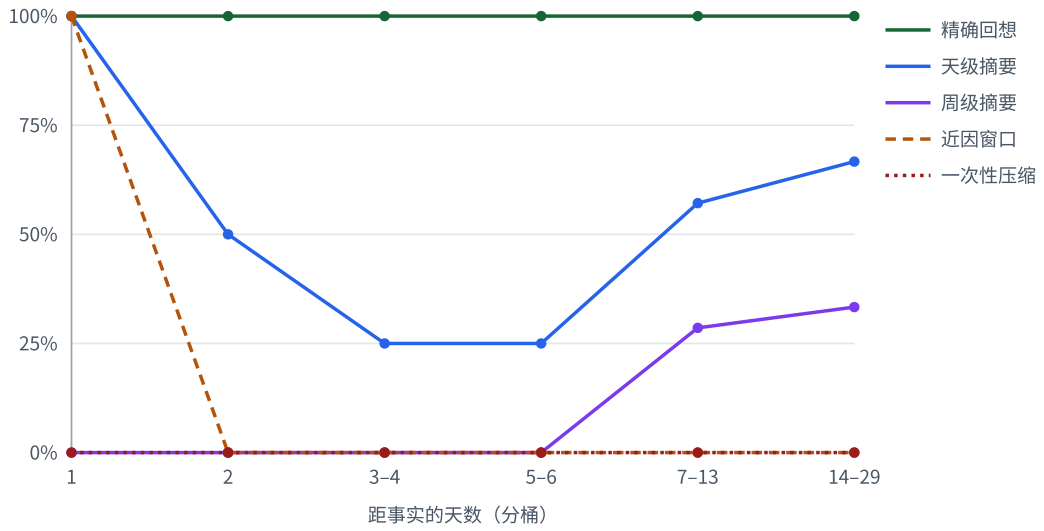


图 1. 各臂的召回保留率 vs 滞后（seed 7）。精确回想全程平在 100%——它**不随事实变老而衰减**。近因基线在滞后一天时满分、从第二天起归零，是断崖而非斜坡。一次性压缩在每一档都是零，**包括最近的窗口**。两条摘要臂会丢信息但**非单调**下降，因为摘要在生成时即定形、之后不随年龄改变（§3.2c）。线条以实线/虚线区分，颜色丢失时仍可辨。

三点观察，按「有多出乎意料」递减排列。

（a）近因基线的跌落是断崖，不是衰减。 它在滞后 1 天时满分，从第 2 天起为零。窗口容纳 20 条、trace 每天产生约 10 条事件，边界因此落在约 2 天处。跨越这条边界时 **内容本身没有任何变化**——那段信息只是不在目标文本里。

（b）一次性压缩在所有档位都是零，包括滞后 1 天。 这是反对破坏性压缩的最强单条结果：它不仅丢失陈旧细节，**连最近的细节也丢**。在我们的实现里，压缩过程把每个窗口改写成 转述（「就 X 做了一项决定」），因此在滞后 1 天时选项字母同样不出现。该实现是**刻意** 破坏性的——保真度正是被测属性——更早的一版保留了决策原句、得了 100%，而经检查它 根本没有压缩，只是在丢弃填充内容。

（c）分层存储的损失是压缩损失，不是时间衰减。 精确回想在每一档都保持 100%， 且**跨 seed 零方差**（六个桶全部 std 0.0）。语义回想并不随滞后单调下降——14–29 天的 78% 反而**高于** 7–13 天的 55%——因为摘要生成一次后不会随年龄改变。它丢的是压缩在 生成时丢掉的东西；**时间不是变量**。

方差一列很有信息量。精确回想是确定性的（每桶 std 0.0，因为读取器在解析引用链）。 语义回想的离散来自 **LLM 采样**：对同一窗口用 `recompute` 重新总结两次，分别产出 237 与 453 字符，即相同输入下完全不同的文本。这意味着摘要的桶均值是分布的**中心**而非精确值， 也是我们报告 ± 并把单次运行的分桶数字视为指示性的原因。

3.3 消融：断崖确实随容量移动

若（a）是容量效应，移动窗口就必须移动断崖。六档窗口，只用本地后端（不需实例、不调 LLM）； 图 2 画出结果：

top_k	预测断崖（top_k ÷ 10 事件/天）	实测断崖	整体
10	1.0 天	⩽0 天	0.0%
20	2.0 天	⩽1 天	6.2%
30	3.0 天	⩽2 天	12.5%
50	5.0 天	⩽3 天	25.0%
100	10.0 天	⩽5 天	56.2%
200	20.0 天	⩽7 天	93.8%

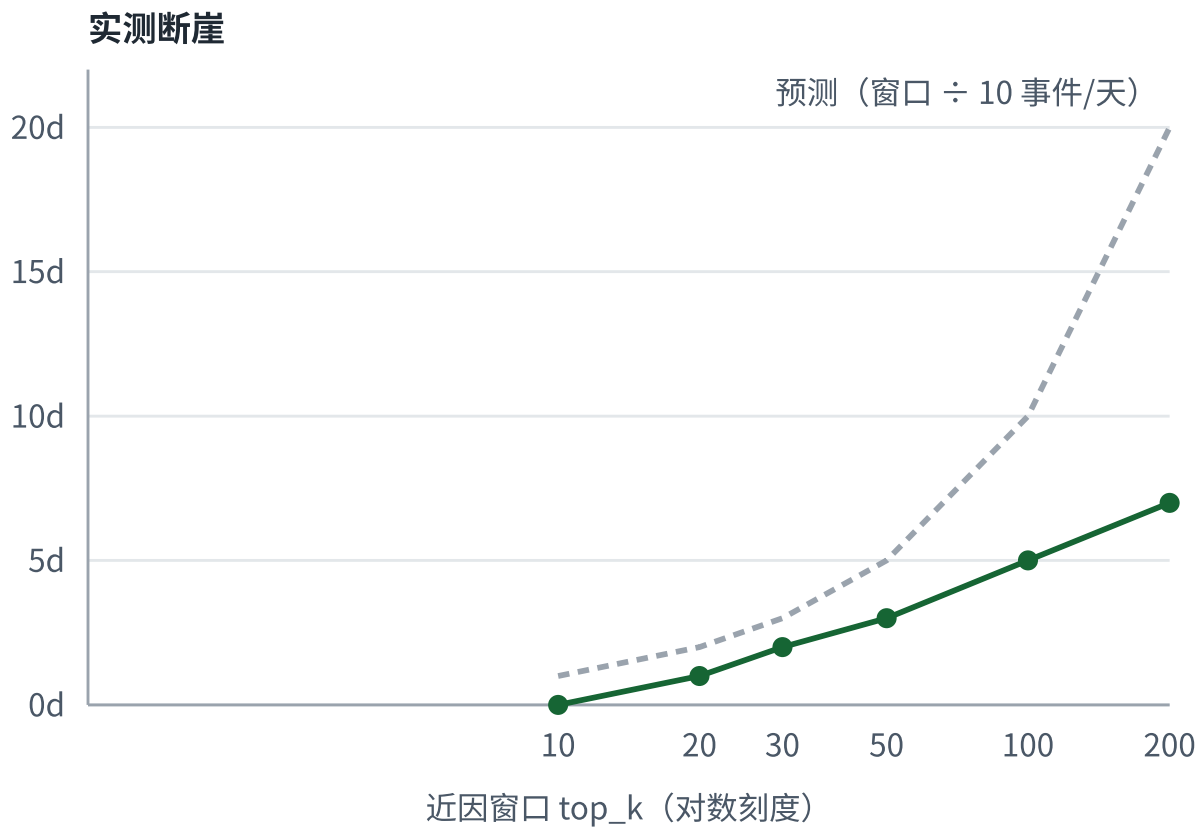


图 2. 消融：实测断崖随窗口容量移动。灰色虚线是预测——容量为 `top_k` 条的窗口覆盖 `top_k ÷ 10 事件/天` 天；绿线是实测。实测略低于预测，因为桶边界稀疏——断崖只能落在边界上，而相邻那一桶要求其中每个探针都答对。

断崖单调移动（0→1→2→3→5→7）。若它不管窗口多大都停在 2 天，容量解释就被推翻， 差异就必须用抽取器或探针构造来解释。实测略低于预测，原因有二且都不是缺陷：桶边界稀疏 （断崖只能落在边界上，而中间那档要求其中**所有**探针都正确），以及 `top_k=200` 的两个 错误来自 29 天的探针，20 天的窗口覆盖不到。

（d）粗层不是细层的廉价版，而是另一种取舍。 直觉的说法会是「读周摘要，拿到更短的答案」。 测量结果相反：

臂	准确率	每问 client token	每问 server token
`chronomem day`	53.1%	**88**	37 500
`chronomem week`	18.8%	**195**	43 750

周级臂每问读的文本更多，而不是更少，答对率却只有约三分之一。 原因在单条响应里看得见。 问第 1 天的 `ZR01` 选了什么，天级摘要写的是 `ZR01 (bob): 选择方案 A`；周级摘要写的是：

> Day 1 至 Day 6 期间，针对 ZT02、ZR01、ZV04、ZU03… 等项目的 … 模块，依次选定 Option A/B/C

十个引用号、一句话，而**引用号到选项的对应关系没有了**。周文本比天文本更长（764 对 343 字符）， 因为它必须提到六天的标识符——但它提的是**名单，不是答案**。

因此粗层的成本画像不是「输出更小」，而是**「摘要的摘要」在压缩时丢掉了条目与其取值之间的配对**， 同时保留了条目的名字。这与 §5.2 在真实数据上的发现一致——标识符比附加其上的数量与机制活得久—— 也意味着：一个需要某条具体事实的客户端，并不是在为更粗的答案付更少的钱，而是在付**差不多的钱 买一个可能根本不包含该事实的答案**。

分层设计仍是月级召回得以可能的原因——精确回想在每一档都是 100%，天级也保住了一半以上—— 但收益**不来自粗摘要便宜**，而来自它下面那些细层仍然可用。

这是 §4 从另一个方向刻画的同一机制。在那里，由**单天**摘要构建的周摘要与其输入逐字节相同 （无压缩、无损失）；在这里，由六天构建的周摘要确实做了合并，并丢掉了条目与取值的配对。 **粗层的保真度取决于其输入有多多样，成本取决于它必须点多少名字**——两者都不由层级位置决定。

---

4. 递归步：保真度取决于输入多样度

4.1 为什么最直觉的测量是错的

我们要刻画的这一步是：*某项目在一周内的摘要 → 该项目的周摘要*。把它测成 *某一天的摘要 → 该周的摘要*是无效的，因为周覆盖五到七天、长约 3.4 倍， 保留率被「目标文本单纯更大」抬高了。

在同一份 24 天样本上两种口径都测：

比较	目标长度	数量保留
天包 vs 周包（范围不对等）	中位 13143 字符 vs 3440	**42.2%**——粗层看起来**更保真**
同项目，一天 vs 该周的对应段	范围不对等	**约 81%**
同项目、同一 7 天窗口（本文采用）	范围对等	见下

第一行就是破绽：一个**粗层比细层更保真**的测量，测的是文本长度，不是压缩。

4.2 真正的结果

来自三个已闭合周的 40 个项目-周对。输入规模是该周内此项目**确实有摘要的天数**。 图 3 画出保留率与输入规模的关系。

输入天数	对数	数量	标识符	因果连接词	长度比
1	29	**90%**	97%	90%	**0.94x**
≥2	11	**58%**	65%	**48%**	**0.53x**
全部（混合）	40	—	—	—	0.82x

单输入的情形不是压缩。 这 29 对里，周摘要与其所依据的天摘要**逐字节相同**—— 在 755、1112、972 字符的文本上实测相似度 1.0000。LLM 不可能逐字复现 755 个汉字； 模型是选择**原样复述输入**。所以那里的「90% 保留」意味着「**没有发生压缩**」， 而不是「压缩是无损的」。

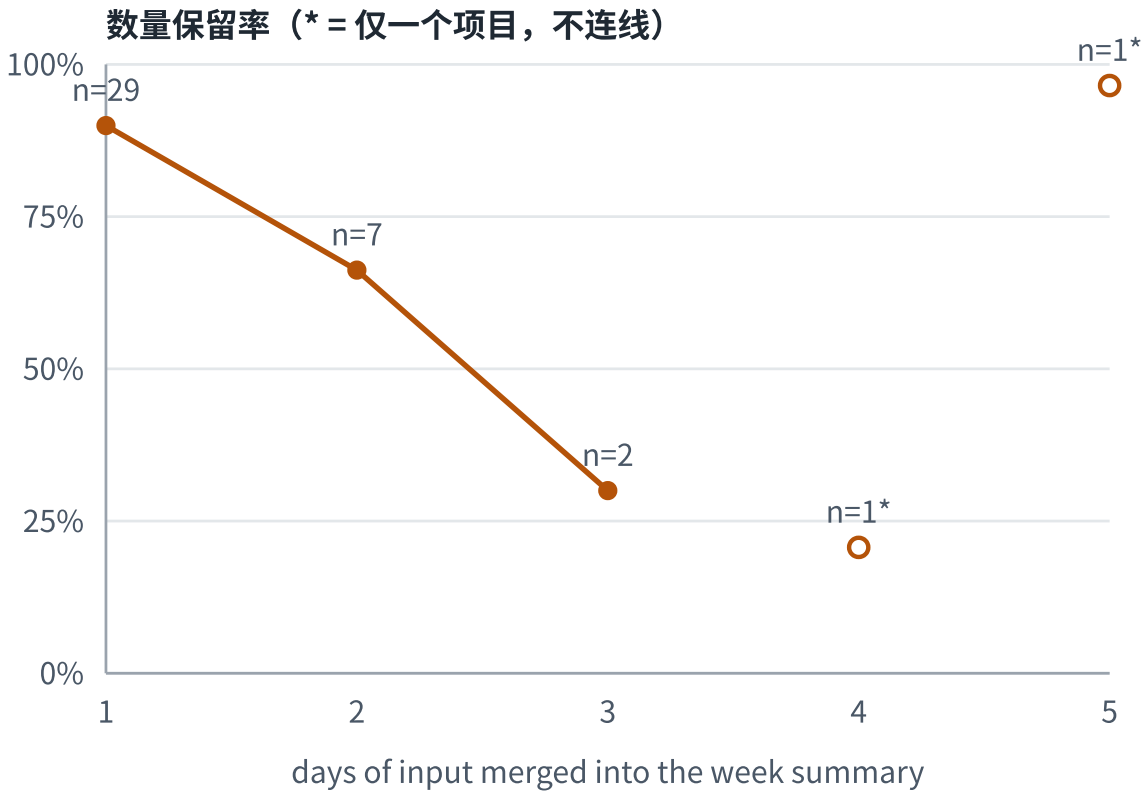


图 3. 递归步按「被要求合并多少」成正比地损失。数量保留率 vs 合并进该项目周摘要的输入天数。单天点（n=29）接近 100%，因为该步在**复述**输入而非压缩它；需要合并的天数越多、保留越低。标 `*` 的点只含一个项目，绘制但不连线，否则单个观测会把曲线带偏。

压缩只在输入多样时才出现。输入达到两天及以上时， 该步把文本减半（0.53x）， 数量保留降到 58%、因果连接词降到 48%。趋势随输入规模延续：

输入天数	长度比	数量保留
1	0.94x	90%
2	0.64x	66%
3	0.15x	30%
4	0.20x	21%

4.3 推论

（a）跨输入规模平均会抬高保真度。 把 29 个单输入对混进平均， 长度比从 0.53x 变成 0.82x， 表观保真度显著上升。任何对递归总结器的评估都**必须按输入多样度分层**， 否则会把「什么都没做」的那些情形当成成功来报。

（b）两跳差一个数量级。 `原始 → 天` 压缩 3.1–4.2 倍； `天 → 周` 压缩 0.53 倍。 原因不是层级而是输入：第一跳吃的是原始条目，第二跳吃的是已经被压过一次的摘要。 因此「每层压缩比固定为 k」并不是这套架构的属性。

（c）**这条链不是定点压缩器**。 保真度由「该步被要求合并什么」决定，所以 「每一层压缩 k 倍」这类说法不成立。成立的是：**这条链从不丢失它没被给到的信息， 且压缩量正比于输入的多样度**。

---

5. 真实语料上什么活了下来

5.1 盲测：无 LLM 路径足以召回精确事实

设计。 我们先自己从工作日志里读出六个事实（一次 OOM 事故的根因与两项修复；一次重试/退避策略的变更及其理由；一次产品改名以及有意不改的部分；一处关于两个组件关系的文档订正；三天摘要缺失的根因；一次模型 A/B 对照及其 prefill 吞吐）。然后派一个 agent，其**唯**一信息源是经由 MCP 的记忆服务——不访问仓库、不上网、不用先验知识——向它提出这六个问题。

结果：6/6 全对，且全部经由 `exact` 路径（沿引用链下钻，回路中不调用 LLM）答出。答案包含精确的操作细节：常驻内存 30–60 MB 对瞬时峰值 217–240 MB、堆上限由 256 MB 提到 1 GB、prefill 吞吐 317 对 267 tok/s，以及改名时有意保留的那几个键及其理由。

这正是 position 论文关于成本不对称的主张，现在是被**测量**而非被论证的：**单靠无 LLM 路径就能召回两个月前的具体操作性事实**。 它不表明所有值得保留的东西都被保留了——只表明被保留下来的那些是可召回的。

同类的一个更小的测试在 36 小时合成 trace 上给出：摘要路径 3/6，而精确路径 6/6——即摘要丢掉了原始条目仍保留的事实。

5.2 链上先丢什么，按什么顺序

在同一语料上，最先损失的是**量化证据**与**因果机制**；专有名词与标识符存活最久。以下案例每一条都通过取回三个层级并检索该事实的词项核实过：

- *数量。* 一次打包清理把清单从 244 个文件收到 101 个。两个数字在天级与周级摘要中都不存在，留下的是「0.4.1 版已发布」。一项下载护栏从 1/3 提到 3/3，峰值 RSS 恒定在 284 MiB；周摘要里 `护栏`、`3/3`、`284` 一个都没有。
- *机制。* 「排序比较器拿一个含 `@version` 的标签当主键，于是版本序退化成字典序」，在两个更粗的层级上都只剩**「排序」**这一个词。标识符 `pkgLabel`、`@version` 以及「字典序」全部消失。
- *规则是最脆的。* 可复用的规则（「改内置函数语义前先查 API 文档」）几乎从不以规则形态存活；它们只在一条不相干的待办（「同步 docs-site」）里留下痕迹。

两点限定。其一，**被用来下结论**的数量会被丢掉，而充当**状态或验收口径**的数量常常存活——所以不是「数字会丢」，而是「**证据性的数字会丢**」。其二，这个顺序只在**真正发生压缩**处成立（§4）：单输入情形什么都不丢，因为什么都没压。

5.3 一个运维层面的发现：一次总结失败留下了持久缺口

该部署的周摘要覆盖了 6 个已闭合周中的 4 个。缺失的两周（09-07 与 09-14）并非随机缺席：同一时期的一个调度缺陷（compact 条目超出端点的 32768 token 上下文 → HTTP 400 → 失败 → 无限重试）导致总结器有两天**产出为零条记录**，而工作日志路径不受影响。正常批次在周一 00:10–00:20 生成；那两周从未出现在这个节奏点上。

由于调度器只总结**最近一个已闭合窗口**，缺口持续存在。补算这两周（对每个窗口显式调用一次 `mem/summarize`）把周覆盖恢复到 6 周，并把链式测量从 0/6 可召回提升到 3/6。

我们报告这一点，是因为它是合成基准**无法产出**的那类证据：**记忆系统自身的一次故障，在它自己的记忆里留下了一个可测量、可持久的缺口**，而缺口的形状（恰好限于故障期、天级摘要完好）指认了成因。

---

6. 方法论：两个陷阱与一个负面结果

6.1 陷阱一——一个把结论反转的长度假象

把整天摘要对整周摘要比较，得到的是**周层的保留率高于天层**（数量上 42% 对 29%）。作为一个压缩主张，粗层比它的输入更保真是**不可能**的，而原因是算术：周包长约 3.4 倍、多覆盖六天，所以某个 token 更可能因巧合而出现。把两侧都限制到同一项目同一窗口后，顺序就反了过来（§4）。

推广。 任何在长度与范围不同的文本上算出的保留指标都值得怀疑。破绽是**与机制相矛盾的排序**。

6.2 陷阱二——一个把失败制造出来的抽取歧义

探针原本按（主题，天）定位答案。摘要的散文结论惯常把多个主题与多个选项写进一句话（「keb 与 vun 分别应用了选项 A 与选项 C」），于是取首个选项字母的读取器会返回另一个主题的答案——而那个错误答案是**貌似合理的**，所以看起来不像 bug。分层后端显示的每一次失分都追溯到这一点；改为逐事实的不透明引用号后，该后端得 100%。

推广。 当某个后端的失败集中在一个答案取值上时，先怀疑读取器，再怀疑系统。

6.2b 陷阱三——打包段里返回相邻事实的答案

摘要把一天压成一行是常态：`- Day 1: ZT02 选方案 B (keb)，ZR01 选方案 A (rix)`。取段内首个选项的读取器会用 `B` 回答 `ZR01`。那个字母是同一段里的合法选项，所以这次失分看起来像「答错了」而不是读取器 bug——而且**层级越粗、事实被压得越密，该假象越强**。它让粗层臂在六天以内的每一档都报 0%，而正确选项就在文本里。

修法是位置感知的：定位引用号，读它**之后**的选项（当引用号跟在选项之后时，读它**之前最近**的那一个）。

推广（陷阱二与陷阱三）。 当某个后端的失败集中在一个答案取值上，或当它们只出现在某一层而正确答案明明就在文本里时，**先怀疑读取器，再怀疑系统**。

6.3 负面结果——LLM 判定者夸大了丢失

判断一个事实是否「存活」在机械上并无定义，于是我们在同一批材料上跑了独立的 LLM 判定者。它们的输出自信而详细。在我们深入核验的具体主张中，**两个都错了**：

判定者的主张	核验
同一周的周摘要逐字节相同（同一 md5、15796 字符）	☑ **成立**
周摘要「就是最后一天的天摘要集合」	✕ **推翻**——逐日比对只有 0–1 个分组逐字相同
某天的摘要「完全略去了 chronomem 组；整条因果链与全部数字都没了」	✕ **推翻**——该组存在且完整，包含根因链与 `OOM`、`sseCond`、`CLOSE_WAIT`、`心跳` 等词

两个被推翻的主张朝**同一个方向**偏：断言了一个并不存在的缺失。判定者的**计数**（intact / partial / absent 分布）同样因此不可引用。

判定者仍有一项用处：它们**指向了正确的现象**——周输出逐字节相同这件事是真的，而发现它引出了 §4 按输入规模的分析。但方向与数字必须来自机械的重新测量。

6.4 我们因此依靠什么

§3–§5 中每一条量化主张都来自可重新运行、且输出随仓库归档的脚本：带 seed 的 trace 生成器、五臂运行器、消融、token 存活与递归步测量、以及链式核查。判定者的输出也归档了，但标注为仅作方向参考。工具本身有 31 个回归用例，外加一个变异检查——它刻意以十一种方式破坏实现（包括上述陷阱），如果测试套件没有察觉就判失败。这些是在 §6.1–6.2 的陷阱之后补的，**因为如果测试从未碰到它所声称覆盖的东西，测试通过本身说明不了什么**。

---

7. 局限

- **三个 seed**。 合成结果基于 seed 7/11/23。精确回想在各桶零方差，使其 100% 稳健；摘要数字带有高达 38% 的分桶标准差，应读作中心而非点估计。摘要每桶只有 2–14 个探针。
- **合成 trace**。 机制层面的结论（容量断崖、破坏性压缩）可迁移；依据本 trace 标定的量级不可。
- **真实语料没有标准答案**。 §5 是存活测量与盲测召回，不是准确率。§5.1 表明被记录下来的东西是可召回的；它无法表明重要的东西都被记录下来了。
- **判定者覆盖不完整**。 计划四片判定（24 天），两片产出（12 天）。由于判定者的计数不可引用（§6.3），这只影响定性色彩，不影响所报告的数字。
- **单一部署、单用户、约两个月**。 未验证多租户、鉴权或加密运行；规模是少数几种类型、数千条记录。
- **只测了 `天→周` 递归**。 服务支持更细的层级（1 小时及以下），其行为本文未加刻画。
- **压缩只有一种实现**。 一次性压缩基线是我们自己的；更强的压缩器会改变它的数字，但不会改变那个结构性论断——它无法回答关于已被它丢弃的内容的问题。

---

8. 结论

我们着手执行 position 论文的评估计划，结果发现最有意思的东西关乎**测量本身**。

对照部分是明确的：有界近因窗口在容量以内满分、超出则完全没有，而六点消融显示断崖随容量移动；一次性压缩即使最短滞后也是零分；分层的精确回想在整个跨度上保持 100% 且零方差。position 论文所论证的成本不对称是真实且可测的：单靠无 LLM 路径就答对了 6/6 个关于两个月前操作性事实的盲测问题。

递归总结部分要求我们**两次修正自己的方法**，而修正后的结果比原来的说法更具体。**该步的保真度由输入多样度而非层级决定**。 当某项目只贡献一天时，周摘要与其输入逐字节相同——没有压缩，因而也没有损失；当合并若干天时，文本减半，且一步之内约 40% 的数量与一半的因果连接词消失。「每层压缩比」不是这套架构的属性。在真实数据上，损失的顺序是稳定的：先证据性数字，再机制，标识符活得最久——而**可复用规则的脆弱性是最可能在实践中要紧的模式**，因为规则本应是经验中可泛化的那部分。

最后是一个我们没料到会值得报告的负面结果：独立的 LLM 判定者产出了具体、自信而错误的断言——我们深入核验的两个都断言了并不存在的缺失。我们报告它，是因为同一个失效模式会影响任何依赖模型判断

「这是否存活」的记忆系统评估：**判定者看不见只以摘要形态呈现给它的东西**，而不加小心的评估者同样看不见。

参考文献

（沿用 position 论文的参考文献表。新增引用的制品：）

1. Position 论文：*Chronomem (忆时塔): Time-Layered External Memory as an MCP Service for Agent Recall and Summarization.* AiraXiv 2610.0035.
2. 评估工具与结果归档：实现仓库中的 `chronomem/bench/` 与 `chronomem/bench/results/`。
3. 综合评估文档：`chronomem/docs/evaluation-retention.md`、`chronomem/docs/evaluation-real-corpus.md`、`chronomem/docs/evaluation-baseline.md`。

AI 使用披露

实现、评估工具与本文分析均与一个 AI 助手（DeepSeek）协作完成。独立的 LLM 判定者被用作**反例**：其输出作为负面结果报告于 §6.3，且不是任何量化主张的来源。所有报告的数字都由随仓库归档的脚本产出，并在撰写过程中与该输出逐条复核。